D15A / 26

EXPERIMENT NO:05

AIM: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

1. Dataset Source

The dataset used for this experiment is the **Pima Indians Diabetes Dataset**, obtained from Kaggle.

Source: Kaggle
Link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

The dataset contains diagnostic health measurements used to predict whether a patient has diabetes. It is widely used for machine learning classification experiments.

2. Dataset Description

The dataset contains medical data of female patients of Pima Indian heritage.

Dataset Details

- **Total records:** 768
- **Number of input features:** 8
- **Target variable:** Outcome

Feature Description

Feature	Description	Type
Pregnancies	Number of times pregnant	Numerical
Glucose	Plasma glucose concentration	Numerical
BloodPressure	Diastolic blood pressure	Numerical
SkinThickness	Triceps skin fold thickness	Numerical
Insulin	Serum insulin level	Numerical
BMI	Body Mass Index	Numerical
DiabetesPedigreeFunction	Genetic likelihood of diabetes	Numerical
Age	Age of patient	Numerical

Target Variable

Outcome

- 0 → Non-diabetic
- 1 → Diabetic

Dataset Characteristics

- Structured medical dataset
- Binary classification problem
- No categorical variables present
- Suitable for machine learning classification algorithms

3. Mathematical Formulation of SVM

Support Vector Machine (SVM) is a supervised learning algorithm used for classification tasks.

The goal of SVM is to find the optimal hyperplane that separates data points belonging to different classes with the maximum margin.

The equation of the hyperplane is:

$$w \cdot x + b = 0$$

Where

- w = weight vector
- x = input feature vector
- b = bias term

The distance between the closest data points (support vectors) and the hyperplane is called the **margin**.

$$Margin = \frac{2}{||w||}$$

When data is not linearly separable, SVM uses **kernel functions** such as:

- Linear kernel
- Polynomial kernel
- Radial Basis Function (RBF)

These kernels transform data into higher-dimensional space to enable separation.

4. Algorithm Limitations

Support Vector Machine has several limitations:

1. Training time increases significantly for large datasets.
2. Selecting appropriate kernel and hyperparameters can be difficult.
3. Performance may decrease if the dataset contains noisy data.
4. SVM models are less interpretable compared to simpler models.

Therefore, proper **feature scaling and parameter tuning** are required for optimal performance.

5. Methodology / Workflow

Step-by-Step Workflow

1. Dataset collected from Kaggle.
2. Data loaded using Pandas.
3. Dataset inspected for missing values and structure.
4. Input features and target variable separated.
5. Dataset split into:
    - **80% Training data**
    - **20% Testing data**
6. Feature scaling applied using **StandardScaler** to normalize feature values.
7. Support Vector Machine classifier implemented using **SVC()**.
8. **Hyperparameter tuning performed using GridSearchCV**.
9. Best model selected based on accuracy.
10. Model evaluated using accuracy score, confusion matrix, and classification report.

6. Performance Analysis

The SVM model was evaluated using classification metrics.

Results Obtained

- **Accuracy: 0.7208 (72.08%)**

Weighted Average F1-Score: 0.71

Interpretation

- The model correctly classifies most non-diabetic cases with high recall (0.83).
- Performance for diabetic cases is lower, indicating some misclassification.
- Overall accuracy of **72.08%** shows moderate classification performance on the dataset.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** to improve the model performance.

Parameters Tuned

- **Kernel:** Linear, RBF
- **C (Regularization parameter):** 0.1, 1, 10
- **Gamma:** 0.01, 0.1, 1

The optimal parameters were selected based on cross-validation accuracy, which helped improve model generalization and classification performance.

CODE and OUTPUT:

Step-by-Step Implementation

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

✓ 8.9s

Step 2: Load Dataset

```python
df = pd.read_csv("diabetes.csv")

print(df.head())
```

✓ 0.0s

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Step 3: Check Dataset Information and Missing Values

```python
print(df.info())
print(df.isnull().sum())
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

Step 4: Define Features and Target

```python
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

Step 5: Split Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

✓ 0.0s

```
Training samples: 614
Testing samples: 154
```

Step 6: Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

✓ 0.0s

Step 7: Define Hyperparameter Grid

```
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': [0.01, 0.1, 1],
    'kernel': ['linear', 'rbf']
}
```

Step 8: Train SVM Model using GridSearchCV

```
grid_search = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)
```

✓ 2.3s

```
▸ GridSearchCV

▸ best_estimator_:
    SVC

  ▾ SVC
  ▶ Parameters
```

Step 9: Display Best Parameters

```
print("Best Parameters:", grid_search.best_params_)
```

✓ 0.0s

```
Best Parameters: {'C': 0.1, 'gamma': 0.01, 'kernel': 'linear'}
```

Step 10: Prediction

```
best_svm = grid_search.best_estimator_
y_pred = best_svm.predict(X_test)
```

✓ 0.0s

Step 11: Evaluation

```
print("SVM Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

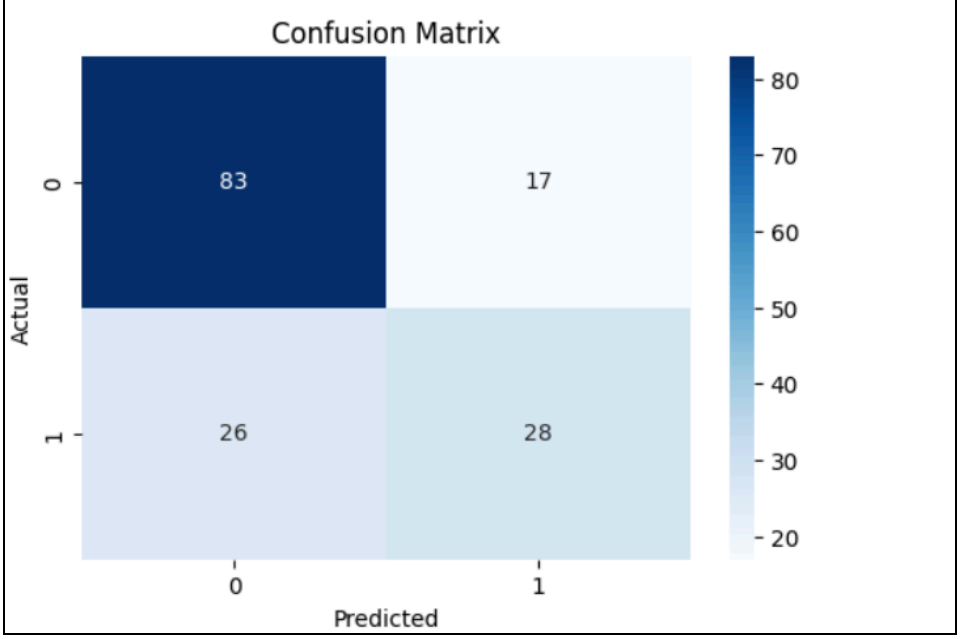
✓ 0.0s

```
SVM Accuracy: 0.7207792207792207

Confusion Matrix:
 [[83 17]
 [26 28]]

Classification Report:
               precision    recall  f1-score   support

           0       0.76      0.83      0.79       100
           1       0.62      0.52      0.57        54

    accuracy                           0.72       154
   macro avg       0.69      0.67      0.68       154
weighted avg       0.71      0.72      0.71       154
```

Step 12: Confusion Matrix Visualization

```python
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

✓ 0.6s



Conclusion

The Support Vector Machine classifier was successfully implemented on the Pima Indians Diabetes dataset. After applying feature scaling and hyperparameter tuning, the model achieved an accuracy of 72.08%. The experiment demonstrates the effectiveness of SVM for medical classification problems and highlights the importance of parameter tuning in improving model performance.